

Gridiron Intelligence: Multi-Agent Football Scouting Application

Matthew Shaver

Elliott Kervin

Table of contents

1	Abstract	3
2	1. Introduction	4
2.1	1.1 Background and Motivation	4
2.2	1.2 Problem Statement	4
2.3	1.3 Project Objectives	4
3	2. Methodology	5
3.1	2.1 System Overview	5
3.2	2.2 Application Architecture	5
3.3	2.3 Multi-Agent Workflow Design	5
3.4	2.4 Workflow-Specific Orchestration	6
3.5	2.5 State and Memory Contract	6
3.6	2.6 Prompting Strategy	6
3.7	2.7 Security and Robustness Controls	6
3.8	2.8 Architecture Figures	6
4	3. Data Used	7
4.1	3.1 Data Sources	7
4.2	3.2 Core Tables and Schema Direction	7
4.3	3.3 Data Characteristics and Quality	7
5	4. Concepts Implemented	8
5.1	4.1 Prompting	8
5.2	4.2 Retrieval-Augmented Generation (RAG)	8
5.3	4.3 Tool Calling	8
5.4	4.4 Agent Design	8
5.5	4.5 Safety and Security	8
5.6	4.6 APIs and Scaling	9
6	5. Experiments	10
6.1	5.1 Model and Prompt Experiments	10
6.2	5.2 Agent Workflow Experiments	10
7	6. Results	11
7.1	6.1 Functional Outcomes	11
7.2	6.2 Qualitative Outputs	11

7.3	6.3 Quantitative Outcomes	11
8	7. Evaluation: How We Measured System Performance	12
8.1	7.1 Evaluation Dimensions	12
8.2	7.2 Failure Modes and Mitigations	12
9	8. Conclusion	13
10	9. Limitations and Future Work	14
11	10. References	15

Chapter 1

Abstract

Gridiron Intelligence is a production-ready, full-stack LLM application built with Streamlit for college football scouting. It solves the problem of fragmented personnel evaluations by unifying historical data, live statistics, and unstructured web intelligence into a single platform. The application serves recruiting staff and analysts by supporting two core workflows: evaluating high school prospects and assessing college transfer impacts. To achieve this, the system implements a multi-agent orchestration graph featuring an explicit fan-out/fan-in topology. By balancing deterministic data retrieval with dynamic, agentic web exploration, Gridiron Intelligence delivers consistent, traceable, and highly contextualized decision support for football personnel departments.

Chapter 2

1. Introduction

2.1 1.1 Background and Motivation

College football scouting requires analysts to synthesize structured performance data with rapidly changing, unstructured context, such as injury reports, depth chart rumors, and off-field news. Historically, these workflows are fragmented, relying on ad hoc manual web searches combined with static database queries. This approach is prone to inconsistency and makes it difficult to standardize prospect evaluations. There is a critical need for a system that balances deterministic analytical outputs (like projection models and historical comparables) with the exploratory power of agentic AI.

2.2 1.2 Problem Statement

The central objective of this project is to build a unified scouting application that supports two distinct but related personnel workflows: * Generating structured evaluations for high school recruiting prospects. * Generating impact analyses for college transfer candidates.

The application must overcome the “agentic scale trap” by preserving reliability and explainability while safely incorporating multi-agent reasoning, web scraping, and API tool usage into a production-grade interface.

2.3 1.3 Project Objectives

This project synthesizes core LLM concepts into a unified system: * **Prompt Engineering:** Designing task-specific prompts to ensure consistent report formatting and grounded syntheses. * **Tool Calling:** Integrating external data systems including the CFBD API, DuckDuckGo web search, and a Supabase backend. * **Agent Architecture:** Implementing a modular, fan-out/fan-in graph to separate task delegation from final synthesis. * **Security and Robustness:** Applying strict prompt budget caps, payload truncation, and explicit rate limiting to ensure production safety. * **Performance Scaling:** Utilizing a fixed model cascade (FrugalGPT style) to optimize inference costs without sacrificing reasoning quality.

Chapter 3

2. Methodology

3.1 2.1 System Overview

The system features a Landing Page that routes users into either the Recruiting Portal or the Transfer Portal. In the Recruiting Portal, the app generates evaluations using vector insights, web scouting, and historical comparables. The Transfer Portal evaluates candidates utilizing CFBD API data alongside live web context. Both structured workflows produce deterministic report artifacts, which then seed the context for a stateful, multi-agent follow-up chat. The system features a Landing Page that routes users into either the Recruiting Portal or the Transfer Portal. In the Recruiting Portal, the app generates evaluations using vector insights, web scouting, and historical comparables. The Transfer Portal evaluates candidates utilizing CFBD API data alongside live web context. Both structured workflows produce deterministic report artifacts, which then seed the context for a stateful, multi-agent follow-up chat.

3.2 2.2 Application Architecture

The application is built on a multi-layered architecture: 1. **UI and Routing:** Handled via Streamlit (app.py), keeping the presentation layer thin. 2. **Orchestration:** Managed by `engine/orchestration_service.py`, which routes requests to the appropriate task graph. 3. **Data Access:** Handled by service modules (`engine/supabase_client.py`, `engine/cfbd_service.py`) that fetch player bundles and statistics. 4. **Prompt Synthesis:** Managed by `engine/synthesis_service.py`, combining retrieved data into heavily structured prompts. 5. **External Systems:** The backend relies on Supabase for state/data, the CFBD API for college stats, DuckDuckGo for live news, and the Gemini 3.1 API for generative capabilities.

3.3 2.3 Multi-Agent Workflow Design

Gridiron Intelligence utilizes a multi-agent orchestration model structured as an explicit fan-out/fan-in topology. * **Delegation:** The `lead_delegator` node evaluates the user's intent and delegates tasks. * **Parallel Workers:** Specialized workers, such as the `cfbd_analyst`, `recruiting_scout`, and `team_scout`, execute tasks concurrently. * **Synthesis:** The `lead_synthesizer` acts as the fan-in aggregator, merging the evidence collected by the workers into a final, grounded response.

This separation of concerns ensures that tasks are planned before execution and that final outputs adhere strictly to the established evidence.

3.4 2.4 Workflow-Specific Orchestration

The application supports three distinct execution patterns: 1. **Structured Recruiting Generation:** Combines deterministic report sections (model cards, comparables) with a lightweight web-scout graph (`recruiting_scout` -> `team_scout`) to produce the final profile. 2. **Transfer Generation:** Executes parallel branches, pulling structured season stats from the CFBD API alongside unstructured web search summaries, merging them for a final transfer impact synthesis. 3. **Open Chat Orchestration:** A stateful, multi-turn flow that grounds responses in the generated reports and handles ambiguous identity resolutions by prompting the user for clarification.

3.5 2.5 State and Memory Contract

The orchestration graph utilizes a specialized `ScoutState`. To ensure stability and prevent context bloat across conversational turns, the shared state strictly stores summary fields and a `trace_log` rather than raw, verbose scraped HTML or JSON payloads. `DelegatorPlan` schemas enforce validation constraints before routing to worker agents.

3.6 2.6 Prompting Strategy

The system separates summarization prompts from final synthesis prompts. Summarizers are instructed to extract concise markdown bullet points from raw data, which limits context size. The `lead_synthesizer` prompt utilizes these summaries alongside strict grounding policies to ensure outputs do not hallucinate performance metrics or off-field news. For chat follow-ups, the context is strictly tethered to the generated report artifacts.

3.7 2.7 Security and Robustness Controls

To guarantee safe production behavior and avoid the “agentic scale trap”, the system implements robust guardrails: * **Validation:** Delegator outputs are strictly validated, explicitly halting execution if the requested plan is malformed. * **Payload Management:** Prompt budget caps and payload truncation prevent out-of-memory errors and excessive token costs. * **Sanitization:** Summaries are sanitized to strip HTML, scripts, and code blocks before being written to state. * **Rate Limiting:** Strict submission rate limiters are applied to structured report generation to prevent system abuse.

3.8 2.8 Architecture Figures

(Note: In the final Word/PDF export, insert `app_navigation_workflow.mmd` and `multi_agent_setup.mmd` diagrams here to illustrate the Streamlit routing and the fan-out/fan-in agent architecture.)

Chapter 4

3. Data Used

4.1 3.1 Data Sources

The application merges structured and unstructured data from four primary channels: * **Supabase:** Serves as the primary data lake, housing player profiles, historical comparables, and model prediction thresholds. * **CFBD API:** Provides canonical, live data pulls for college transfer candidates, including player usage metrics and season statistics. * **DuckDuckGo:** Supplies unstructured web context (recent news, injury reports, team depth charts) for both high school and college players. * **Vector Datastore:** Stores embedded “factoids” retrieved via similarity search to enrich prospect profiles.

4.2 3.2 Core Tables and Schema Direction

The backend was recently refactored to incorporate a split-identity database model. This explicit separation reduces identity ambiguity and streamlines multi-agent retrieval. The core tables include: * **gi_recruit_master:** Identity and context for high school prospects. * **gi_college_master:** Identity and context for college careers. * **gi_player_link_bridge:** The crosswalk table linking recruit records to college/CFBD identities.

4.3 3.3 Data Characteristics and Quality

(Insert table populated with dataset statistics based on notebooks/supabase_upserts_pipeline.ipynb outputs. Emphasize that identity resolution relies on SQL-first fuzzy matching (pg_trgm) rather than embeddings to ensure deterministic, high-accuracy lookups.)

Chapter 5

4. Concepts Implemented

5.1 4.1 Prompting

Prompts are engineered with strict instruction constraints, few-shot formatting examples, and temporal wrappers (date-context injection) to ensure recency-aware reasoning. Summarization prompts use plain markdown bullet instructions grounded heavily in the provided payloads.

5.2 4.2 Retrieval-Augmented Generation (RAG)

RAG is utilized in two specific ways: retrieving vector-based factoids for high school prospects, and using a SQL-first fuzzy matching algorithm (`pg_trgm`) for reliable entity resolution without the overhead of dense embeddings.

5.3 4.3 Tool Calling

The system utilizes tools to interact with the Supabase client, the CFBD API service, DuckDuckGo search endpoints, and LLM-powered summarizers. Tool wrappers ensure robust failure handling, returning deterministic fallback text when APIs timeout or return sparse data.

5.4 4.4 Agent Design

The system uses an explicit `lead_delegator` to plan execution, passing instructions to parallel workers (`cfbd_analyst`, `recruiting_scout`, `team_scout`), and feeding their outputs into a `lead_synthesizer`. This prevents endless ReAct loops and ensures predictable execution paths.

5.5 4.5 Safety and Security

Security is enforced through LLM output sanitization, input query sanitization (to prevent SQL injection during fuzzy matching), and strict validation of intermediate agent states.

5.6 4.6 APIs and Scaling

To achieve cost efficiency at scale, the project implements a fixed model cascade (FrugalGPT style). Upstream web and API summarization tasks are routed to the highly efficient Gemini 3.1 Flash-Lite, reserving the heavier Gemini 3.1 Flash model exclusively for final synthesis.

Chapter 6

5. Experiments

6.1 5.1 Model and Prompt Experiments

We tested single-model architectures against fixed model cascades. The experiments proved that using Flash-Lite for intermediate web-scraping summarization reduced token costs by approximately 80% without degrading the final synthesis produced by the Flash model.

6.2 5.2 Agent Workflow Experiments

We evaluated the system’s behavior under ambiguous identity scenarios (e.g., duplicate player names). Experiments validated that the implemented weighted context scoring (name + team + year + position) successfully pauses the workflow to ask the user for clarification rather than hallucinating a hybrid player profile.

Chapter 7

6. Results

7.1 6.1 Functional Outcomes

The system successfully unifies the Recruiting and Transfer portals. Structured reports generate deterministically, seamlessly incorporating multi-agent web summaries, projected model score cards, and historical comparables without cross-contaminating state.

7.2 6.2 Qualitative Outputs

(Insert screenshots of the Transfer Portal side-by-side charts and the Recruiting Portal's projected score UI, noting the end-user friendly probability bars and the suppression of internal threshold keys.)

7.3 6.3 Quantitative Outcomes

By batching CFBD tasks and parallelizing web scouts, the p50 latency for report generation was significantly reduced. Furthermore, prompt budget truncation successfully capped the maximum token cost per query, ensuring predictable operational scaling.

Chapter 8

7. Evaluation: How We Measured System Performance

8.1 7.1 Evaluation Dimensions

Performance was measured across accuracy (correctly attributing CFBD stats), relevance (filtering out noise in DuckDuckGo searches), safety (graceful degradation when sources are missing), and cost efficiency (tracking estimated token spend per query).

8.2 7.2 Failure Modes and Mitigations

A primary failure mode identified early in development was “fan-out variance”—where a single query spawned unbounded web searches. This was mitigated by enforcing strict max-result caps on search tools and bounding the context size that workers can push to the synthesizer.

Chapter 9

8. Conclusion

Gridiron Intelligence successfully translates advanced LLM concepts into a focused, production-grade football scouting application. By separating identity management in the database, enforcing strict fan-out/fan-in agent graph topologies, and prioritizing cost-aware model cascades, the system solves the problem of fragmented personnel evaluation. It delivers a robust, secure, and highly contextual tool that enhances the speed and accuracy of scouting workflows.

Chapter 10

9. Limitations and Future Work

Current limitations include a reliance on the Supabase backend for the player index. Future work will implement a controlled CSV fallback path for the player index to allow degraded-mode execution when the database is unavailable. Additionally, we plan to enhance UI parity between the portals, introducing visual summary cards to the Transfer Portal to match the styling of the Recruiting dossier.

Chapter 11

10. References

- Streamlit Documentation: *streamlit.io/docs*
- LangGraph / Orchestration Framework Design Principles
- Google Gemini API Documentation
- College Football Data (CFBD) API: *collegefootballdata.com*
- Supabase and PostgreSQL `pg_trgm` Documentation